

# Chapter 9

## Commitment Schemes

**Stefan Dziembowski**

[www.crypto.edu.pl/Dziembowski](http://www.crypto.edu.pl/Dziembowski)

**University of Warsaw**



# Plan

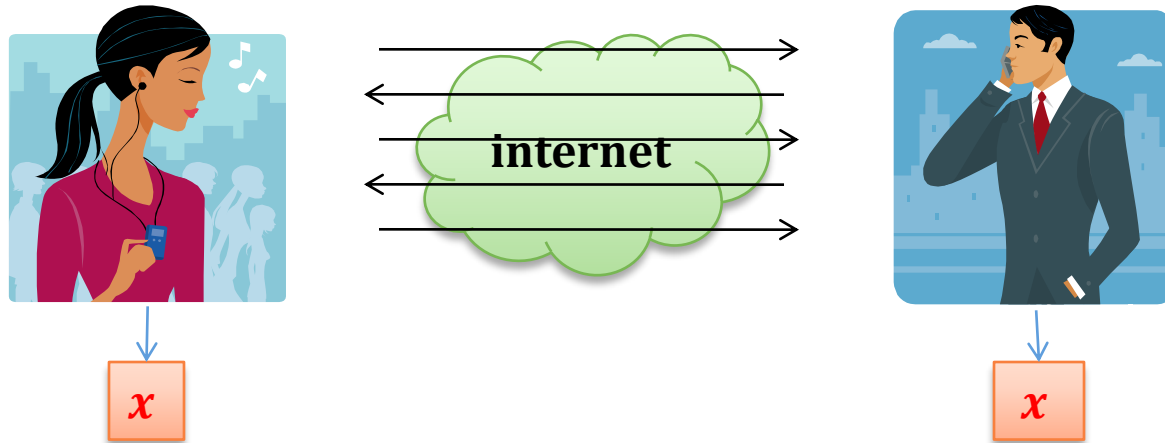


1. Coin-flipping by telephone
2. Commitment schemes

# Coin-flipping by telephone [Blum'81]

privacy and authenticity is not a problem

Suppose Alice and Bob are connected by a secure internet link:



**The goal of Alice and Bob is to toss a coin.**

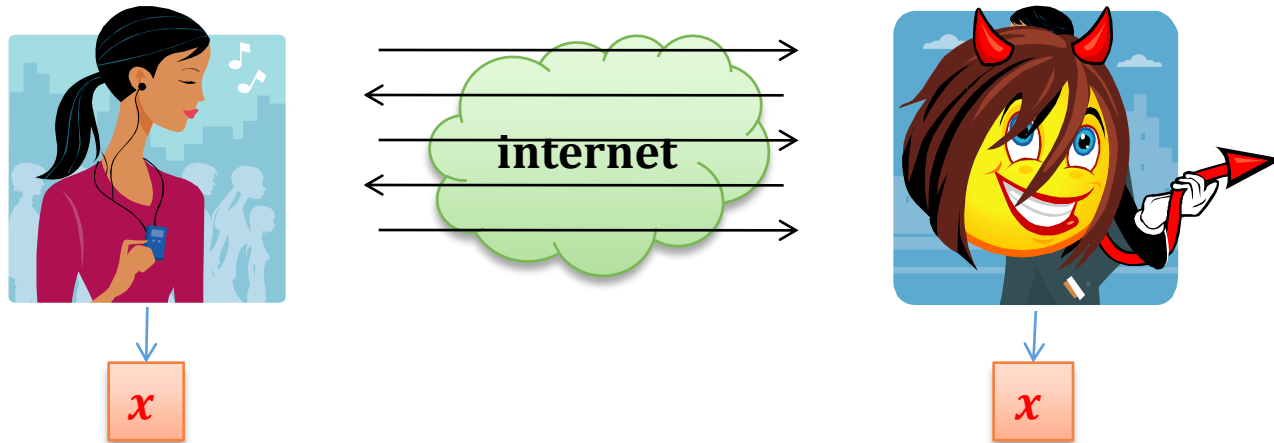
**In other words:**

They want to execute some protocol  $\pi$  in such a way that at the end of the execution they both output the same bit  $x$  distributed uniformly over  $\{0, 1\}$ .

# How to define security? [1/2]

Let us just stay at an informal level...

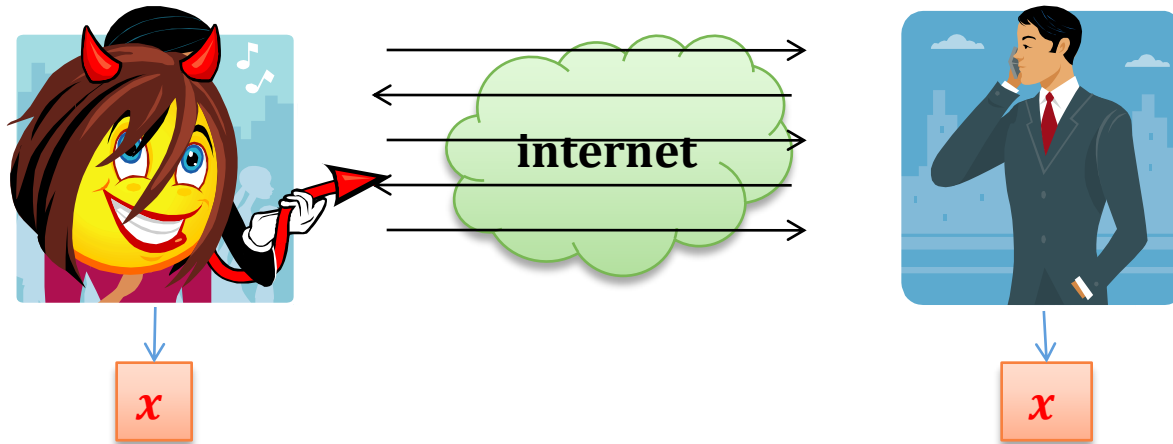
From the point of view of Alice:



even if **Bob** is **cheating** (i.e.: he doesn't follow the protocol):  
if the protocol terminates successfully, then **x** has a  
uniform distribution

# How to define security? [2/2]

The same holds from the point of view of Bob



even if Alice is **cheating** (i.e.: he doesn't follow the protocol):  
if the protocol terminates successfully, then  $x$  has a  
uniform distribution

# Note the difference

Unlike what we saw on the previous lectures:

the enemy can be one of the parties

(**not** an external entity)

A cheating party is sometimes called a **corrupted** party, or a **malicious** party.

We will see many other examples of this later!

# How to solve this problem?

## Idea

Remember the old  
game:

**rock-paper-scissors?**





**Alice**

**Bob**

|                                                                                     |  |  |  |
|-------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|    | draw                                                                               | Alice wins                                                                          | Bob wins                                                                            |
|   | Bob wins                                                                           | draw                                                                                | Alice wins                                                                          |
|  | Alice wins                                                                         | Bob wins                                                                            | draw                                                                                |



# Let's simplify this game

|     |         | Alice      |            |
|-----|---------|------------|------------|
| Bob |         | $A = 0$    | $A = 1$    |
|     | $B = 0$ | Alice wins | Bob wins   |
|     | $B = 1$ | Bob wins   | Alice wins |

In other words: Alice wins iff  $A \oplus B = 0$ .

# Another way to look at it

**Alice**  
has an input **A**

**Bob**  
has an input **B**

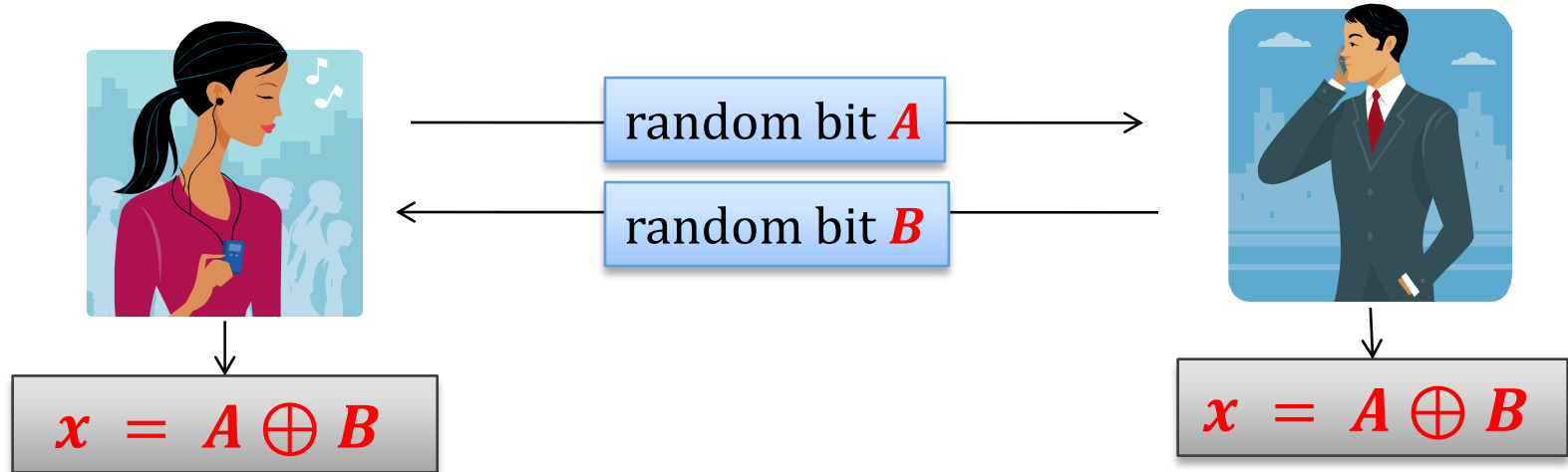


they should jointly compute

$$x = A \oplus B$$

(in a secure way)

# What to do?



## Problem:

$A$  and  $B$  should be sent at the same time

(e.g. if  $A$  is sent before  $B$  then a malicious **Bob** can set  $B := x \oplus A$ , where  $x$  is chosen by him).

# How to prevent this?

Seems hard:

the internet is not synchronous...

A solution:

**bit commitments**

# Plan

1. Coin-flipping by telephone

2. Commitment schemes



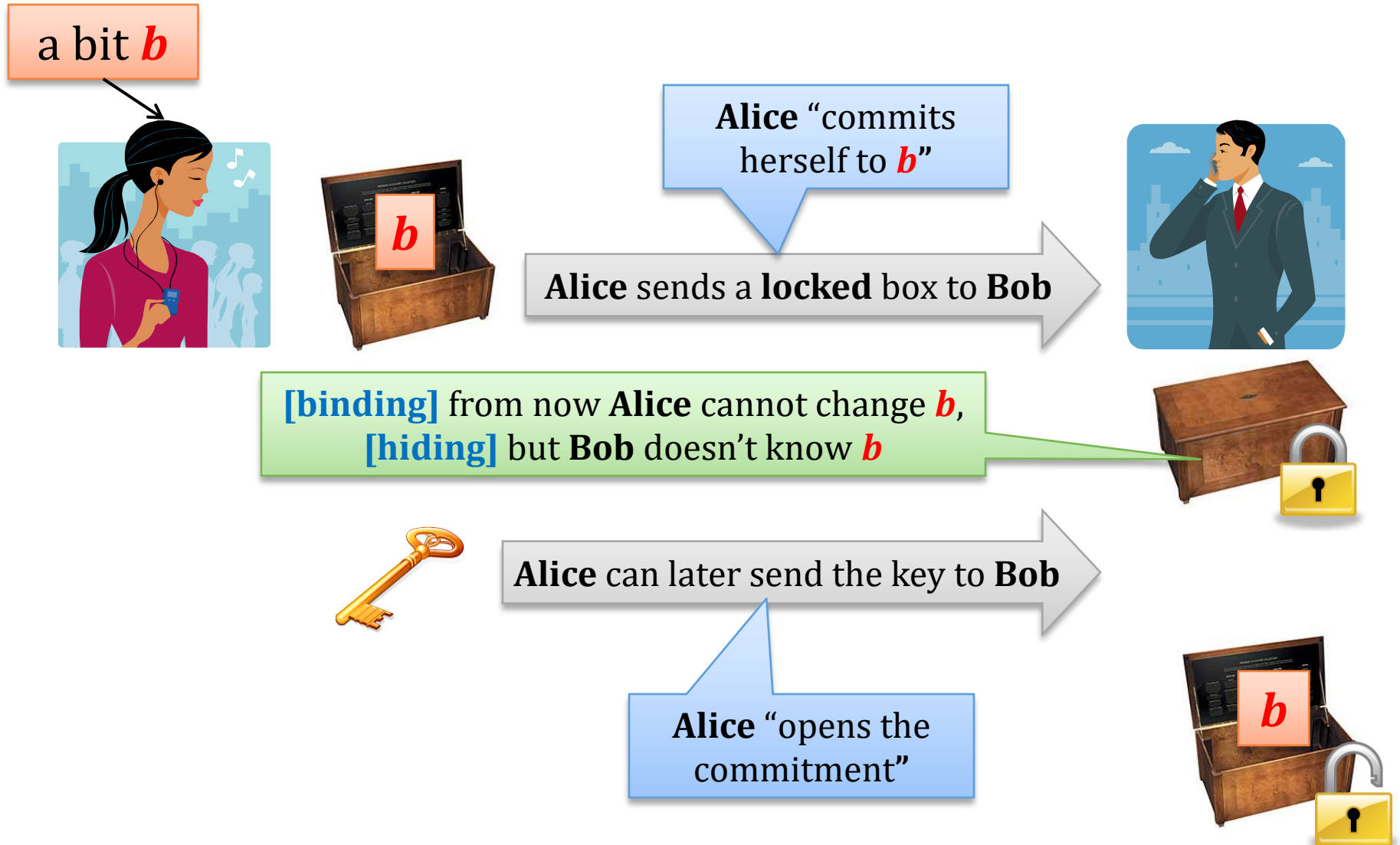
1. definition

2. construction based on QRA

3. construction based on discrete log

4. construction based on PRG

# Commitment schemes – an intuition



# Commitment schemes – a functional definition

A **commitment scheme** is a protocol executed between Alice and Bob consisting of two phases: **commit** and **open**.

In the **commit** phase:

- **Alice** takes some input bit  **$b$** .
- **Bob** takes no input.

In the **open** phase:

- **Alice** outputs nothing
- **Bob** outputs  **$b$** , or **error**

# Security requirements - informally

## [binding]

After the **commit** phase there exists at most one value ***b*** that can be open in the **open** phase.

## [hiding]

As long as the **open** phase did not start **Bob** has no information about ***b***.



# How to define security formally?

Not so trivial – remember that the parties can misbehave arbitrarily.

**We do not present a complete definition here.**

(The hiding property can be defined using the “**indistinguishability**” principle.)

The definition depends on some options.

1. What is the computational power of a **cheating Alice**?
2. What is the computational power of a **cheating Bob**?

# The computational power of the adversary

If a cheating Alice can be infinitely powerful, we say that the protocol is **unconditionally binding**.

Otherwise it is **computationally binding**.

If a cheating Bob can be infinitely powerful, we say that the protocol is **unconditionally hiding**.

Otherwise it is **computationally hiding**.

Of course, to be formal we would need to introduce a security parameter...

# Unconditionally hiding and binding commitment schemes do not exist

## Proof (intuition)

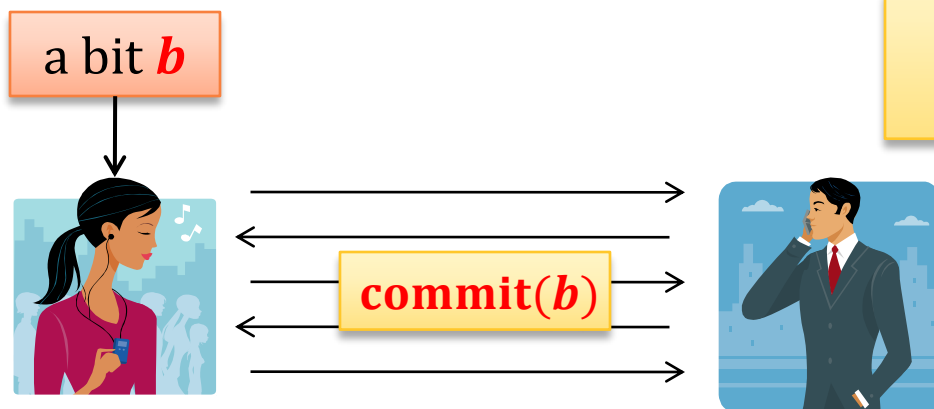
### There are two options:

1. there exists a way to open **1 - b**, or

in this case “infinitely powerful”  
Alice can cheat

2. there doesn't exist such a way

in this case “infinitely powerful”  
Bob can learn **b**



# So, how does it solve the coin-flipping problem?

chooses a random  
bit  $A \leftarrow \{0, 1\}$



commits to  $A$



chooses a random  
bit  $B \leftarrow \{0, 1\}$

sends  $B$

opens  $A$

output  
 $A \oplus B$

output  
 $A \oplus B$

# Problem

**Alice** can always refuse to send the last message.

This is unavoidable (there has to be the **last message** in the protocol).

But they can use a convention:  
if **Alice** didn't send the last message – she lost!

# Plan

1. Coin-flipping by telephone

2. Commitment schemes

1. definition

2. construction based on QRA

3. construction based on discrete log

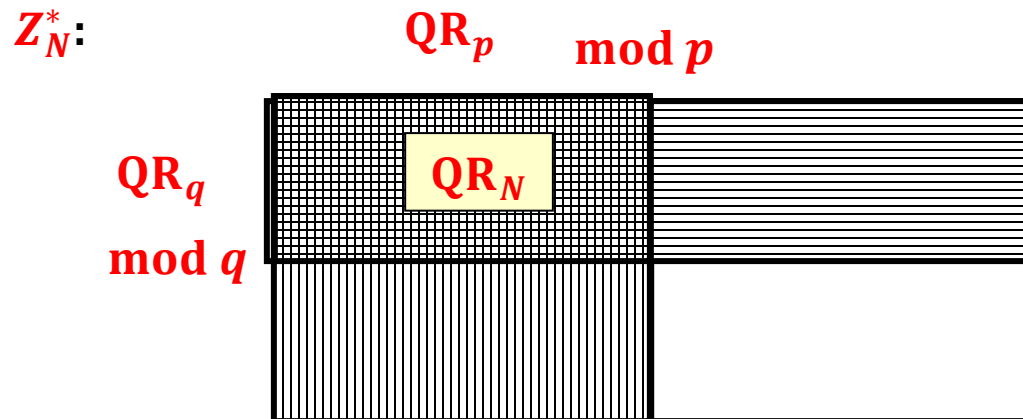
4. construction based on PRG



# Remember quadratic residues modulo an RSA modulus?

$$\text{QR}_N = \{x^2 \bmod N : x \in \mathbb{Z}_N^*\}$$

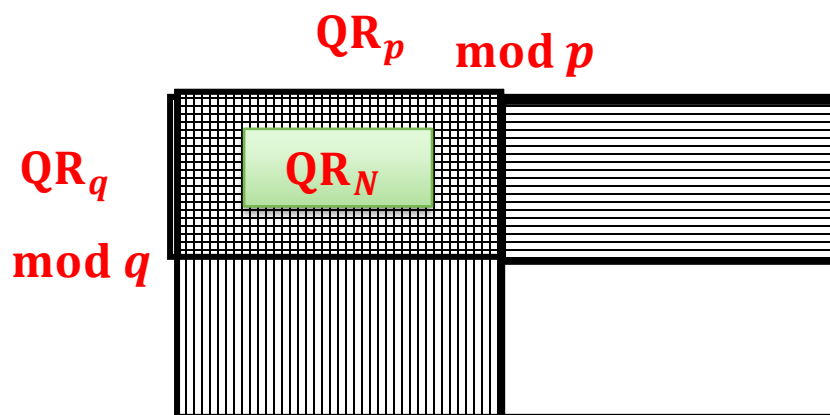
Fact: For  $N = pq$  we have  $|\text{QR}_N| = |\mathbb{Z}_N^*| / 4$ .



# Jacobi Symbol

for any prime  $p$  define  $J_p(x) := \begin{cases} +1 & \text{if } x \in \mathbf{QR}_p \\ -1 & \text{otherwise} \end{cases}$

for  $N = pq$  define  $J_N(x) := J_p(x) \cdot J_q(x)$



$J_N(x) :=$

|    |    |
|----|----|
| +1 | -1 |
| -1 | +1 |

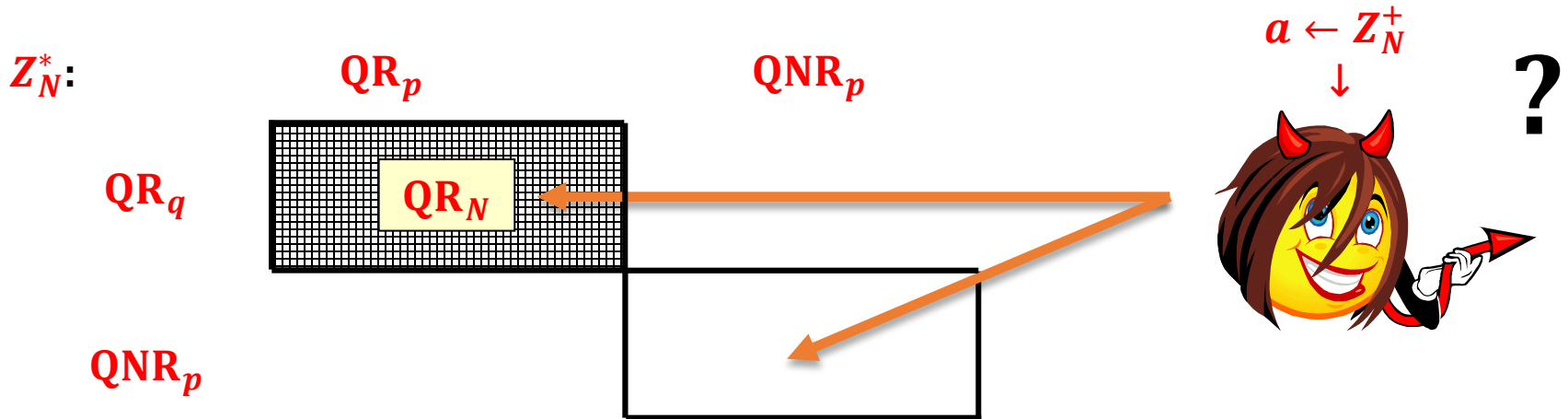
It is a subgroup of  $\mathbf{Z}_N^*$

$$\mathbf{Z}_N^+ := \{x \in \mathbf{Z}_N^* : J_N(x) = +1\}$$

**Jacobi symbol can be computed efficiently!**  
(even in  $p$  and  $q$  are unknown)



# Quadratic Residuosity Assumption



## Quadratic Residuosity Assumption (QRA):

For a random  $a \leftarrow Z_N^+$  it is computationally hard to determine if  $a \in QR_N$ .

**Formally:** for every **polynomial-time** probabilistic algorithm  $D$  the value:

$$\left| P(D(N, a) = Q_N(a)) - \frac{1}{2} \right|$$

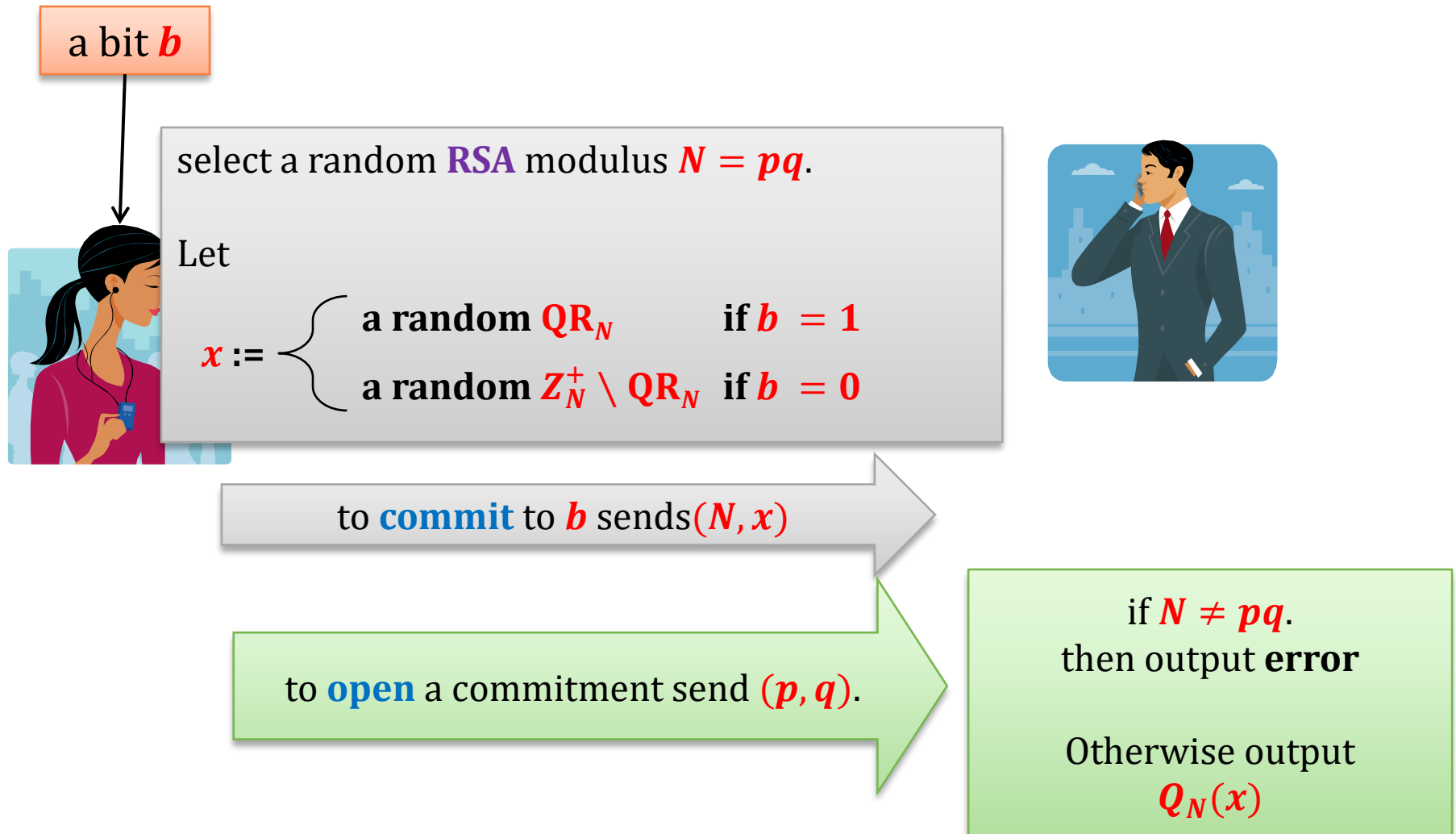
(where  $a \leftarrow Z_N^+$ ) is **negligible**.

Where a predicate  $Q_N: Z_N^+ \rightarrow \{0, 1\}$  is defined as follows:

$Q_N(a) = 1$  if  $a \in QR_N$

$Q_N(a) = 0$  otherwise

# A construction based on QRA



# This commitment scheme is unconditionally binding

Why?

Suppose **Alice** has sent  $(N, x)$  to **Bob**.

What can **Bob** output at the end of the opening phase?

There exists the following options:

- $N$  is not an **RSA** modulus – in this case **Bob** will always output **error**,
- $x \in \mathbf{QR}_N$  – in this case **Bob** can only output **0** or **error**,
- $x \notin \mathbf{QR}_N$  – in this case **Bob** can only output **1** or **error**.

This commitment scheme is computationally hiding, assuming QRA holds

### Proof (intuition)

To distinguish between  $b = 0$  and  $b = 1$  a malicious **Bob** would need to distinguish  $QR_N$  from the other elements of  $Z_N^*$  ...

# Plan

1. Coin-flipping by telephone
2. Commitment schemes
  1. definition
  2. construction based on QRA
  3. construction based on discrete log
  4. construction based on PRG



# A construction based on discrete log

$G$  – a group in which discrete log is hard

$g$  – a generator of  $G$

a bit  $b$

Select a random

$$y \leftarrow Z_{|G|}$$

Let

$$x := \begin{cases} g^y & \text{if } b = 0 \\ s \cdot g^y & \text{if } b = 1 \end{cases}$$

Select  $s$  – a random element of  $G$

$G, g, s$



to commit to  $b$  send  $x$

to open a commitment send  $y$ .

if  $x = g^y$  output  $b = 0$   
if  $x = s \cdot g^y$  output  $b = 1$

This commitment scheme is **computationally binding**, assuming that the discrete log is hard in  $G$

### Proof (intuition)

To be able to open the commitment in two ways, a **cheating Alice** needs to know  $y$  and  $y'$  such that there exists  $x$  such that

$$g^y = x = s \cdot g^{y'}$$

But this means that  $g^{y-y'} = s$ . So, she would know the discrete log of  $s$ .

# This commitment scheme is unconditionally hiding

Why?

Because  $x$  is just a random element of  $G$ .



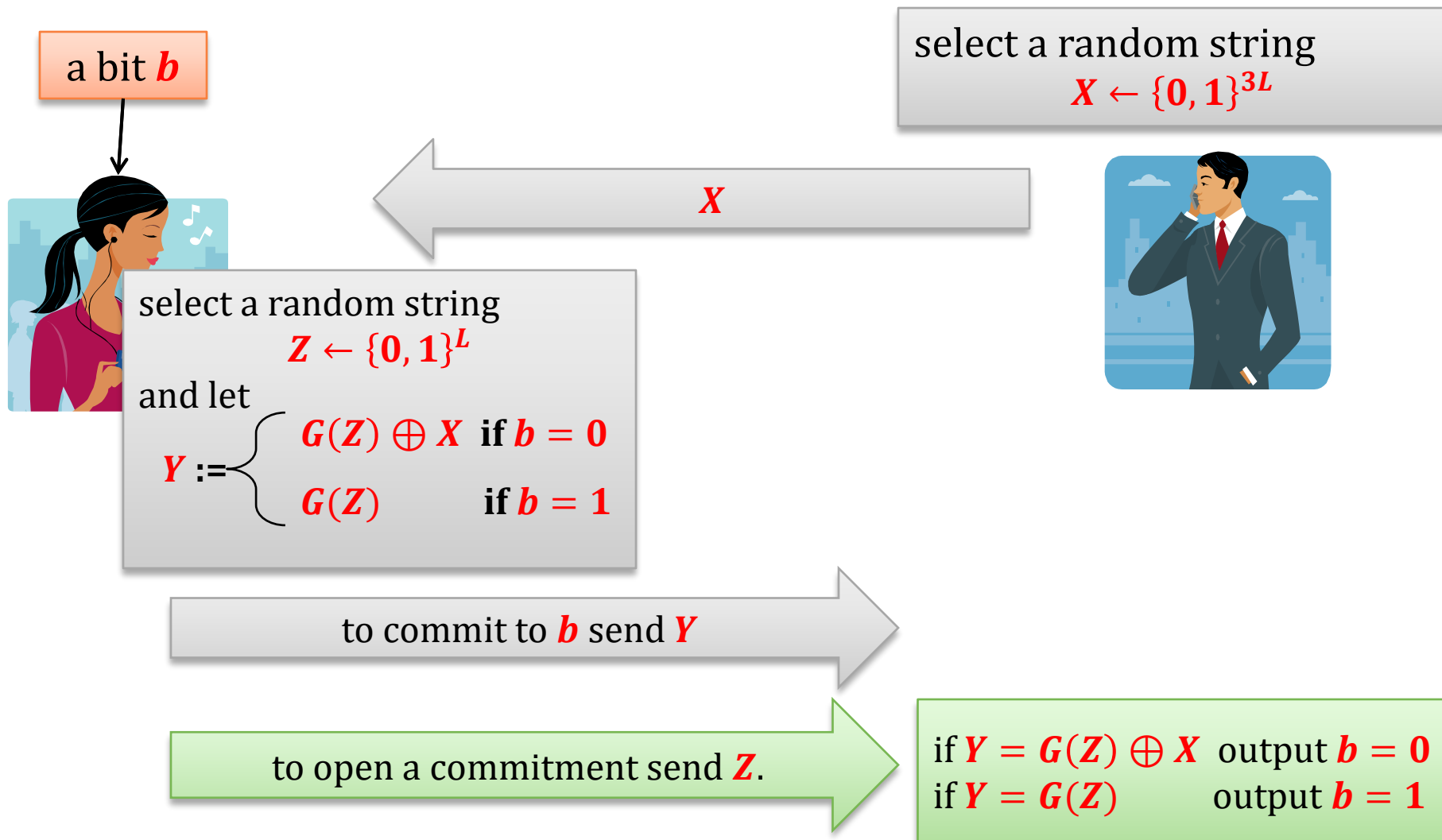
# Plan

1. Coin-flipping by telephone
2. Commitment schemes
  1. definition
  2. construction based on QRA
  3. construction based on discrete log
  4. construction based on PRG



# A construction based on PRGs [Naor'91]

$G: \{0, 1\}^L \rightarrow \{0, 1\}^{3L}$  a PRG



# This commitment scheme is unconditionally binding

## Proof (intuition)

To be able to open the commitment in two ways, a **cheating Alice** needs to find  $Z$  and  $Z'$  such that there exists  $Y$  such that:

$$G(Z) \oplus X = Y = G(Z')$$

This means that  $G(Z) \oplus G(Z') = X$ .

How many  $X$ 's have the property that

**there exist  $Z$  and  $Z'$  such that  $G(Z) \oplus G(Z') = X$ ?**

By the counting argument: at most  $(2^L)^2 = 2^{2L}$ .

Therefore, the probability that a **random  $X \in \{0, 1\}^{3L}$**  has this property is at most

$$\frac{2^{2L}}{2^{3L}} = 2^{-L}.$$

**QED**

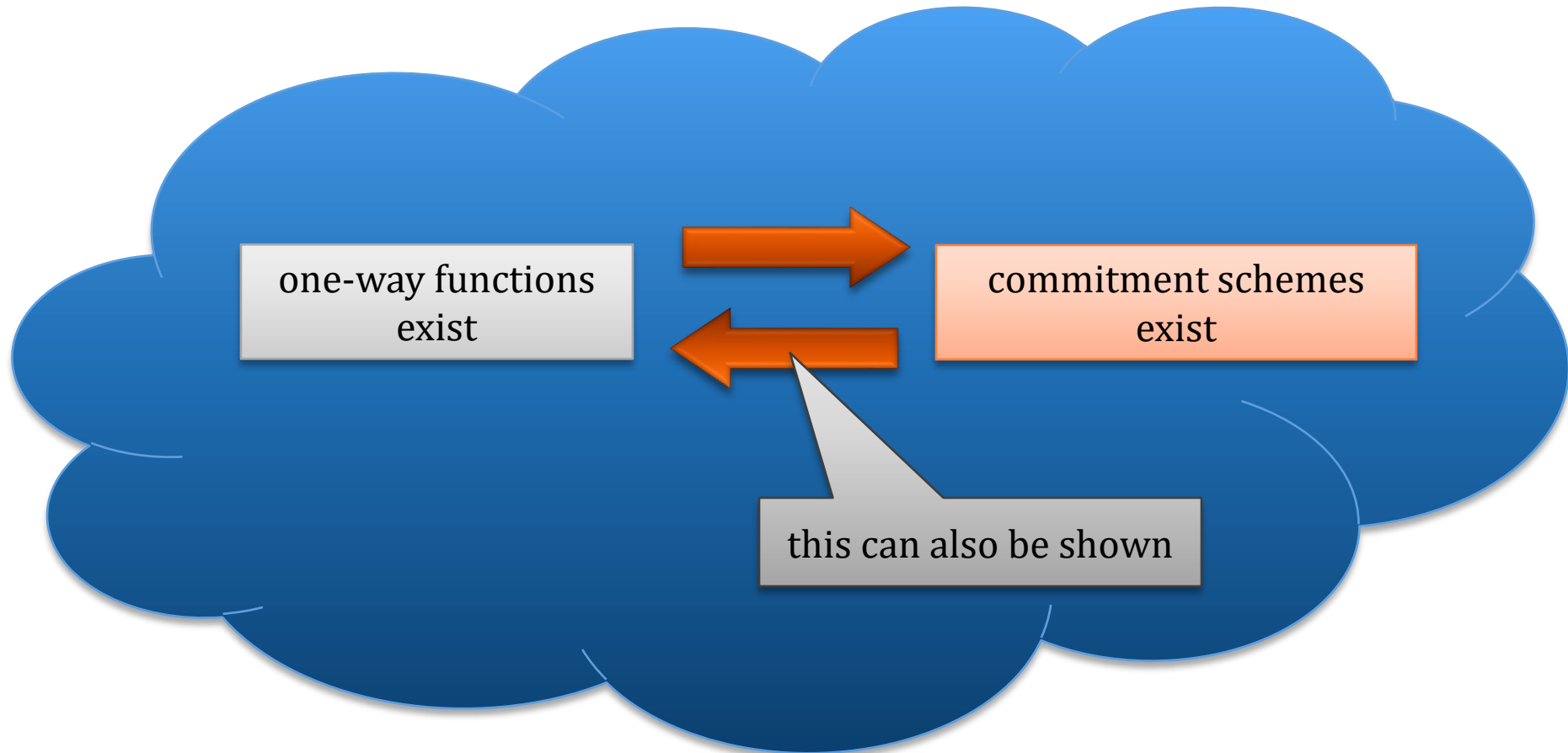
This commitment scheme is computationally hiding, assuming  $G$  is a secure PRG

### Why?

Obviously, if, instead of  $G(Z)$  Alice uses a completely random string  $R$ , then the scheme is secure against a cheating Bob.

If a scheme behaved differently with  $R$  and with  $G(Z)$ , then a cheating Bob could be used as a distinguisher for  $G$ .

# Moral



Commitment schemes are a part of **Minicrypt**!

# String commitment

How to commit to a longer string  $\mathbf{x} = (x_1, \dots, x_n)$ ?

Just commit to every  $x_i$  separately.

To open the commitment, open each commitment to  $x_i$ .

(**Bob** accepts only if all the openings were **ok**)

©2023 by Stefan Dziembowski. Permission to make digital or hard copies of part or all of this material is currently granted without fee *provided that copies are made only for personal or classroom use, are not distributed for profit or commercial advantage, and that new copies bear this notice and the full citation.*